

Prototipo de sistema de navegación con modelo predictivo de la posición GPS con redes LSTM

Salvador Pérez Lago

28 de mayo de 2026

Índice

1. Introducción	3
2. Prototipo físico	3
2.1. Diseño	3
2.2. Funcionamiento	4
3. Modelo predictivo LSTM	7
3.1. Tratamiento de datos	7
3.2. Visualización de datos	7
3.3. Entrenamiento de modelos: redes LSTM	8
3.4. Evaluación: Generación y visualización de rutas predichas vs rutas reales	9
A. Esquema de conexiones	11
B. Información sobre los sensores	12
B.1. Magnetómetro: Adafruit MMC56x3	12
B.1.1. Especificaciones principales	12
B.1.2. Cálculo de orientación (Heading)	12
B.1.3. Calibración	13
B.1.4. Funciones adicionales	14
B.1.5. Consideraciones de diseño	14
B.1.6. Modo de operación	14
B.1.7. Enlaces de interés	15
B.2. Sensor Barométrico y de Temperatura: BMP390	16
B.2.1. Especificaciones principales	16
B.2.2. Modo de operación	16
B.2.3. Funciones adicionales	16
B.2.4. Consideraciones de diseño	16
B.2.5. Enlaces de interés	16
B.3. Acelerómetro y Giroscopio: SKU SEN0386	17
B.3.1. Especificaciones principales	17
B.3.2. Funciones adicionales	17
B.3.3. Enlaces de interés	17
B.4. Módulo GPS: GP-735T	18

B.4.1.	Especificaciones principales	18
B.4.2.	Funciones adicionales	18
B.4.3.	Conexión y configuración	18
B.4.4.	Consideraciones de diseño	19
B.4.5.	Enlaces de interés	19
B.5.	Módulo de Tarjeta SD: DFR0229	20
B.5.1.	Especificaciones	20
B.5.2.	Funciones adicionales	20
B.5.3.	Consideraciones de diseño	20
B.5.4.	Enlaces de interés	20

1. Introducción

En este informe se describe el proceso de diseño e implementación tanto de la parte hardware como software de un prototipo de sistema de navegación predictivo. El objetivo principal del proyecto es el de crear un sistema que, ante la falta de señal GPS, sea capaz de predecir de manera exacta su geolocalización, haciendo uso de la información de diversos sensores inerciales y ambientales y de modelos predictivos de tipo LSTM.

Para lograr este objetivo debe diseñarse un primer prototipo que permita recopilar una buena cantidad de datos reales donde se incluya tanto la información de los sensores como la posición GPS real del sistema. Una vez se tienen esos datos, comienza la etapa de tratamiento y limpieza de los mismos, para prepararlos de cara a poder utilizarlos para entrenar un modelo que permita predecir la variación de la geolocalización en función de las medidas de los sensores. Por último, se debe implementar el modelo hardware con el modelo predictivo, para realizar pruebas en entornos reales que demuestren el correcto funcionamiento del sistema.

2. Prototipo físico

2.1. Diseño

La primera tarea del proyecto propone la obtención de datos sensóricos, sobre los que se realizará más tarde el entrenamiento de los modelos predictivos. Para ello, se diseña un prototipo que integra distintos sensores con un microcontrolador **Arduino UNO R4 WiFi**. Los sensores elegidos son los siguientes:

- Magnetómetro **Adafruit MMC56x3**: mide campos magnéticos en 3 ejes y cálculo de *heading* respecto al norte magnético.
- Sensor de presión y temperatura **Adafruit BMP390**
- Sensor acelerómetro, giroscopio y de ángulo **SKU SEN0386**: aceleración lineal, aceleración angular y posición angular en 3 ejes.
- Receptor GPS **GP-735T**
- Módulo de tarjeta SD **DFR0229**

Aparte de los sensores se han incorporado 3 luces LED y un transistor que funcionan de la siguiente manera:

- LED blanco: Este LED permite conocer el estado de funcionamiento del módulo GPS, ya que no tiene sentido recopilar datos si no hay señal GPS ya que estos serían datos sin *target*, inútiles para entrenar modelos. Además, es muy útil durante el arranque del prototipo ya que el GPS puede llegar a tardar un minuto o dos en calentarse y comenzar a funcionar correctamente. Cuando la señal GPS es correcta, el LED se enciende, y puede volver a apagarse/encenderse durante el funcionamiento según se pierda o recupere la señal.

- **LED ROJO:** Este LED se enciende de inicio y se apaga si la rutina de *setup* de todos los módulos y sensores es correcta (esto incluye, por ejemplo, que en el módulo SD haya alojada una tarjeta formateada correctamente en formato **FAT**). En caso de éxito se apaga y el prototipo comienza a recolectar datos. Si en algún momento alguno de los sensores falla en la lectura de sus datos, el LED vuelve a encenderse.
- **LED amarillo:** Este LED cambia de encendido a apagado cada vez que se realiza una escritura de datos en la SD. De esta forma, como los datos se loguean en *chunks*, es más sencillo parar el programa al final de una ruta sin perder todo ese *chunk* de información.
- **Transistor:** El transistor permite que el propio microcontrolador pilote el funcionamiento del sensor barométrico. Durante las pruebas se han detectado problemas de inconsistencia con ese sensor, perdiendo su conexión con la placa Arduino durante runtime. La única solución que válida que se ha encontrado es retirar la alimentación durante un instante para acto seguido volver a conectarlo. Con el fin de automatizar esta tarea, la alimentación a ese sensor está controlada a través de un transistor, pilotado por una de las salidas digitales del microcontrolador. En caso de detectar que hay un problema de conexión con ese sensor, el LED rojo se enciende, y la salida digital deja de alimentar la base del transistor, cortando la alimentación al sensor. Tras un breve periodo, se vuelve a alimentar la base y se comprueba si el sensor se ha reiniciado con éxito. De ser así, se apaga el LED rojo y el programa prosigue su funcionamiento normal, y en caso contrario se vuelve a intentar el proceso de reinicio con el transistor nuevamente.

Todos estos elementos han sido incorporados según lo descrito en el esquema de conexiones disponible en el Apéndice [A](#). Además, en el Apéndice [B](#) se puede encontrar un apartado donde se describen las características técnicas y algunos datos de interés de los distintos sensores.

2.2. Funcionamiento

El objetivo principal del prototipo es el de loguear información de los sensores a una tarjeta **micro SD** alojada en el módulo **DFR0229**. Este proceso se realiza con una frecuencia de escritura de $1Hz$. Sin embargo, esta frecuencia puede no resultar representativa para todos los sensores, por lo que la toma de los datos se realiza mediante el cálculo de la media móvil de las diferentes magnitudes con distintas frecuencias en función de la naturaleza de los sensores. Una vez se realiza una escritura de datos, comienzan de cero los cálculos para el siguiente logueo, *i.e.* que la información recopilada en cada ventana de tiempo de $1s$ es independiente del resto. Los distintos sensores están configurados para funcionar de la siguiente manera:

- El módulo GPS tiene una frecuencia de trabajo de $1Hz$. Una vez por segundo se realiza una medida de la longitud y latitud de la posición del prototipo, siendo estas dos variables el objetivo a posteriori de las tareas de predicción mediante redes neuronales (la variable *target*). Aunque no se usan, también se recoge la información de velocidad, altura o *heading* que se obtiene de los satélites.

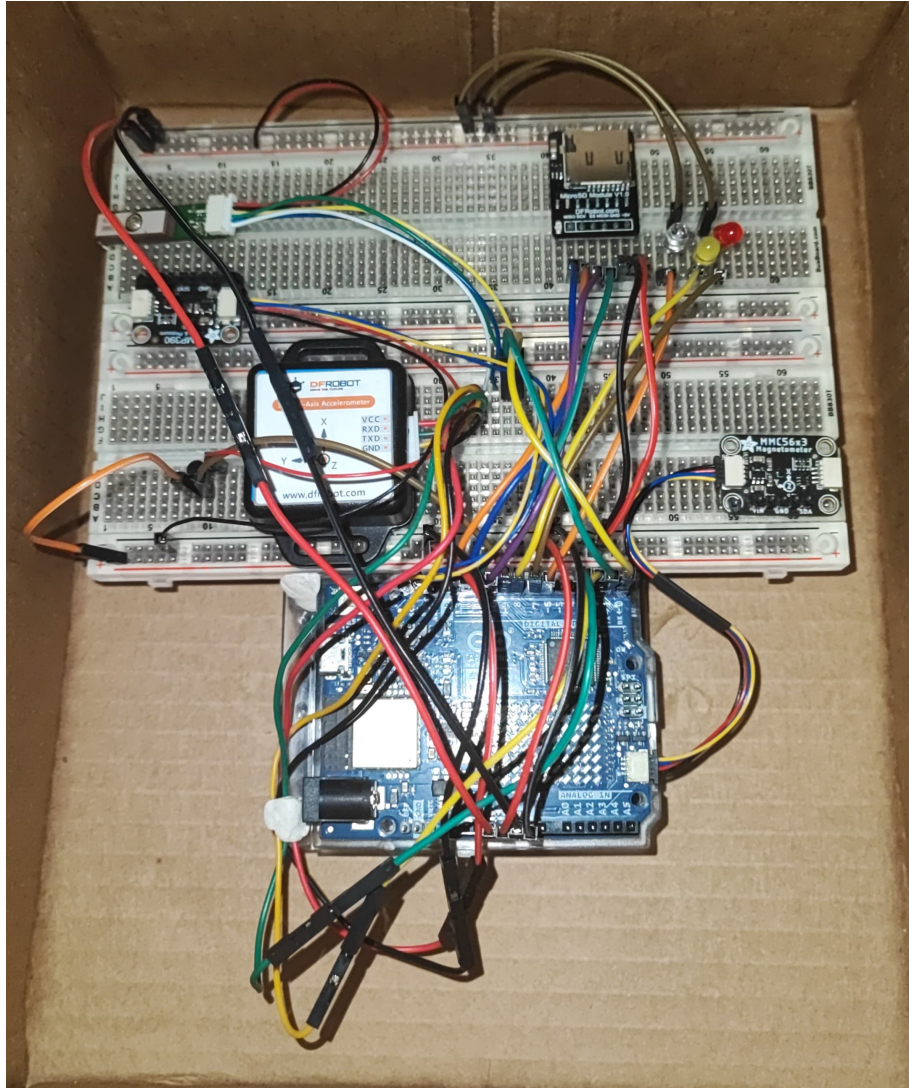


Figura 1: Prototipo físico montado

- El sensor barométrico y de temperatura permite obtener lecturas de la temperatura ambiental, de la presión atmosférica y también, a partir de esta última y en función de una variable que debe ser configurada manualmente en el código (presión atmosférica al nivel del mar en el lugar y momento donde vayan a realizarse las medidas), la altura respecto al nivel del mar. En el caso que nos ocupa, estas variables no presentan un ratio de cambio demasiado elevado, por lo que se realizan medidas con una frecuencia de $5Hz$.
- El sensor acelerómetro es el más crítico en esta tarea. Tomar medidas con una frecuencia demasiado baja puede resultar en que esas medidas discretas no sean representativas del movimiento real que ha sucedido durante un periodo de tiempo, por lo que es importante que la frecuencia de lectura de este sensor sea lo más alta posible para aproximarse a la función continua real. En este caso, la frecuencia que se ha logrado es de $15Hz$, que es el máximo que se ha conseguido. Se han realizado pruebas donde se han eliminado los procesos de lectura del resto de sensores y de logueo, para dedicar toda la capacidad del microcontrolador a realizar lecturas de este sensor, pero no se ha logrado superar la frecuencia indicada, por lo que seguramente se trate de la máxima frecuencia teórica de trabajo del sensor aunque en su *datasheet* se indiquen frecuencias mayores. Por otro lado, también cabe la posibilidad de que representar el movimiento en un periodo dado mediante la media de la aceleración durante ese mismo periodo no sea representativo del todo y que lo correcto fuese calcular mediante integración la velocidad o la posición, aunque el cómputo disponible en este caso tampoco es suficiente para realizar estas operaciones en runtime a una frecuencia adecuada.

Una vez terminado el periodo de $1s$, toda la información sensórica recopilada es logueada en formato `csv` a un fichero previamente creado (al iniciar el programa se genera el fichero con las cabeceras) en la tarjeta SD alojada. Tras este paso, se reinician las variables en las que se alojan las medias de las medidas de los sensores y comienza el proceso de nuevo. Con el fin de agilizar el proceso se ha creado un *buffer* en el que se van almacenando líneas en la memoria del microcontrolador y, al llegar a un número N (es un parámetro del código), se realiza el volcado de todo ese *chunk* de líneas a la tarjeta. Así se disminuye el número de veces que se deben realizar escrituras en la tarjeta, ahorrando algo de tiempo de ejecución.

Tras cablear físicamente el prototipo y volcar el programa en el microcontrolador, se alimenta el mismo mediante USB y se procede a realizar la recogida de datos. Para ello, se diseñan y realizan diversas rutas en coche en las que se puedan apreciar variaciones de las variables medidas (aceleración lineal y angular, altura, posición GPS, *heading*, ...).

3. Modelo predictivo LSTM

3.1. Tratamiento de datos

El primer paso para poder entrenar modelos predictivos que funcionen correctamente es el tratamiento y limpieza de los datos. En este caso, se dispone de archivos `csv` individuales, cada uno de los cuales se corresponde con una ruta distinta, donde las filas contiguas representan datos recogidos en puntos también contiguos en el espacio real, *i.e.* los datos recopilados son secuenciales dentro de cada archivo, pero cada uno de los archivos representa rutas individuales.

Con esos primeros datos se realiza el siguiente proceso en el programa `csv_processing.ipynb`:

- Creación de `delta_lat` y `delta_lng`: Estas 2 nuevas variables serán el *target* para los modelos predictivos. Se tratan de la diferencia entre los valores absolutos de `latitud` y `longitud` de una fila a la siguiente. Por este motivo, la primera fila, al no tener valores previos de estas dos variables, genera valores 0,0 para los deltas calculados, por lo que debe descartarse de cara al entrenamiento de los modelos. También se calcula de forma análoga la variable `delta_alt`, que no ha llegado a usarse pero podría ser interesante de cara a un futuro para realizar predicciones que también incluyan el eje Z.
- Selección de variables: Algunas de las variables recopiladas por el prototipo pueden no ser útiles para entrenar a los modelos, por lo que se eliminan, resultando en un vector con la siguiente estructura:

`[Acc_X; Acc_Y; Acc_Z; Gyro_X; Gyro_Y; Gyro_Z; Tilt_X; Tilt_Y; Tilt_Z; Mag_X; Mag_Y; Mag_Z; Head; Temp; Press; Lat; Lng; delta_Alt; delta_Lat; delta_Lng]`

- Estandarización de las variables: Con el fin de que todas las variables tengan magnitudes similares para evitar problemas al entrenar el modelo, se realiza una normalización *MinMax* siguiendo la fórmula $X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$, teniendo en cuenta para cada una de las variables los valores máximos y mínimos teóricos de los distintos sensores.

3.2. Visualización de datos

Con el fin de facilitar el estudio de los datos generados en el apartado anterior, se crea el notebook `plotter.ipynb`, en el que se han implementado una serie de funciones que permiten visualizar la evolución de las variables *target* en cada uno de los archivos `csv` o dibujar las rutas sobre un mapa real, sobre el que se plotean cada uno de las posiciones GPS que generan la ruta.

Al realizar plots de las variables objetivo, se ha descubierto que existen picos de variación extraños que no parecen corresponderse con movimientos naturales y que podrían ser más debidos a problemas de funcionamiento o a la baja precisión del GPS. Si se visualizan los mapas con las posiciones reales de GPS, puede verse que los puntos contiguos no siempre presentan distancias uniformes entre si, provocando esos picos en los diferenciales. Esto podría deberse también a cambios de velocidad al realizar las rutas, aunque no parece que esto pudiese explicar por si solo todos los picos. También es posible ver que

la precisión del GPS no es del todo correcta, ya que en ocasiones, a pesar de mantener la forma de los caminos del mapa real, las posiciones GPS recopiladas presentan un pequeño *offset* respecto a ellos.

Por ese motivo, se realiza un suavizado de los caminos mediante el uso de filtros de Kalman, generando rutas suavizadas que se plotean en las gráficas junto a las rutas reales. Aunque estos caminos presentan mucha más uniformidad en la distribución de los puntos, también sufren de *offset* respecto de los caminos reales del mapa topográfico. Además, estos caminos suavizados no podríamos decir que se corresponden con la información de los sensores, sería "falsear" la variable objetivo, por lo que se descarta su uso para entrenar a los modelos.

3.3. Entrenamiento de modelos: redes LSTM

El siguiente paso tras la preparación de los datos es el entrenamiento del modelo predictivo. En este caso, por la naturaleza secuencial de los datos se ha decidido utilizar redes neuronales de tipo LSTM. Se han realizado pruebas con otras arquitecturas, pero ninguna de ellas ha logrado igualar los resultados obtenidos por las LSTM. Entre las opciones que se han descartado se encuentran las LSTM bidireccionales, las GRU y las redes densas convencionales.

Para poder trabajar con las LSTM, no solo se necesita alimentar a la red con una fila de datos de cada vez, sino que es necesario crear secuencias que incluyan la fila actual más las N filas anteriores. Por este motivo, los modelos no pueden realizar predicciones para los primeros N puntos de cada ruta de forma eficaz. Por este motivo, esa variable N es un parámetro del programa, que permite preparar los datos de las rutas en secuencias de ese tamaño para el entrenamiento.

La separación de los datos en entrenamiento y test se debe realizar de forma manual. Aquellas rutas que se vayan a utilizar para entrenar al modelo deben trasladarse a la carpeta `‘/data/train’` y las que vayan a usarse para la validación deben ir a `‘/data/test’`.

El proceso de implementación y entrenamiento del modelo es el siguiente:

- **Definición de la arquitectura:** Se ha implementado una red neuronal secuencial con una capa LSTM de 64 neuronas, seguida de una capa densa de 32 neuronas y una capa de salida de 2 neuronas que predice los cambios en latitud y longitud (`delta_Lat` y `delta_Lng`). La función de activación para la capa densa intermedia es `leaky_relu`, lo que ayuda a mejorar el aprendizaje en comparación con activaciones lineales simples. La capa de salida no tiene función de activación, pues los valores previstos son deltas que pueden ser tanto positivos como negativos.
- **Compilación del modelo:** Para entrenar el modelo, se ha usado el optimizador Adam con una tasa de aprendizaje adaptativa. La función de pérdida seleccionada es el *error cuadrático medio* (MSE), que es adecuada para problemas de predicción como este, y como métrica de evaluación se incluye la tasa de `accuracy`.
- **Entrenamiento:** El modelo ha sido entrenado durante un total de 100 épocas con un tamaño de batch de 64, utilizando como datos de entrenamiento aquellos

almacenados en la carpeta `train`, y validando el rendimiento del modelo sobre los datos de la carpeta `test`. Este entrenamiento generó una historia de aprendizaje que se graficó para analizar la evolución del error a lo largo de las épocas.

- **Guardado del modelo:** Tras el entrenamiento, el modelo fue guardado en el formato Keras, permitiendo su posterior carga y uso en pruebas y evaluaciones adicionales.

3.4. Evaluación: Generación y visualización de rutas predichas vs rutas reales

Para la evaluación final del modelo se ha implementado una función que permite visualizar las rutas reales y las rutas predichas a partir de los archivos `csv` de test:

- **Cálculo de la ruta real:** Para cada archivo de prueba, se toma el primer punto GPS absoluto como punto de inicio y se aplica la suma acumulativa de `delta_Lat` y `delta_Lng` para reconstruir la ruta real.
- **Cálculo de la ruta predicha:** De manera similar, se utiliza el punto inicial GPS y se aplican las predicciones del modelo sobre las secuencias de datos para generar la ruta predicha.
- **Gráfica comparativa:** Se representa la ruta real en color azul y la ruta predicha en color rojo. Esto permite observar visualmente la precisión del modelo y evaluar las desviaciones que puedan existir entre ambas rutas.

Esta visualización es esencial para evaluar el rendimiento práctico del modelo ante situaciones donde el GPS pierde señal, ya que muestra la capacidad del sistema para mantener una trayectoria coherente en base a las predicciones de las redes neuronales LSTM.

Hasta el momento, los resultados obtenidos sugieren que el modelo LSTM es capaz de adaptarse a los patrones de desplazamiento reflejados en los datos de los sensores, ya que la forma de las rutas predichas es bastante similar a las de las rutas reales. Sin embargo, como se mencionó previamente, el modelo aún experimenta cierta desviación a lo largo del tiempo, generando un *offset* que se va acumulando con cada predicción, por lo que la separación aumenta a medida que pasa el tiempo. En la siguiente imagen puede apreciarse algunos ejemplos sobre las rutas de test:

Con el fin de solucionar este problema de separación entre las rutas reales y las predichas se ha intentado implementar un sistema de filtro de Kalman que funcionase en conjunto junto al modelo LSTM. La solución planteada para capaz de corregir de forma correcta el offset si de vez en cuando se simula que la señal GPS se recupera en alguno de los puntos, pero se descarta porque no serviría en caso de que la señal GPS no esté disponible durante un periodo de tiempo largo, divergiendo de lo que sería el objetivo principal del proyecto.

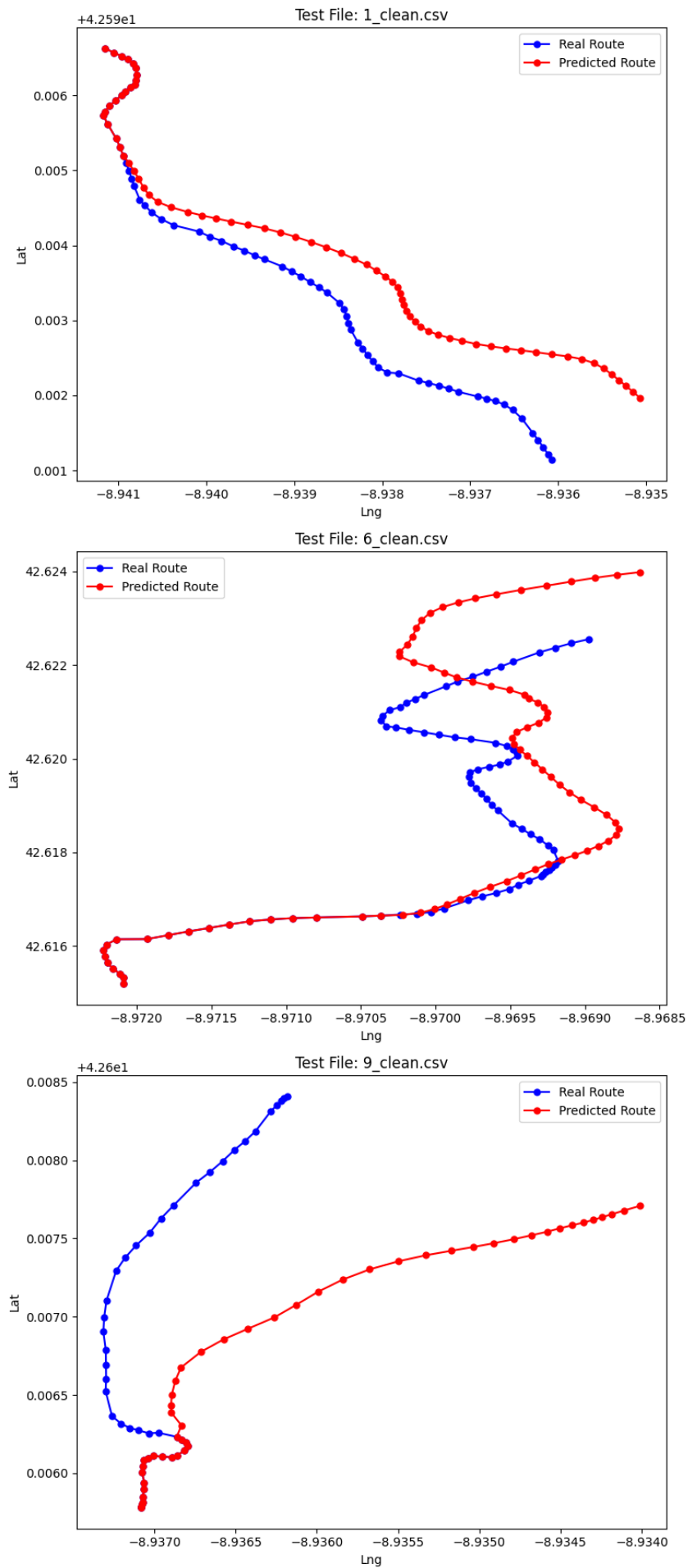
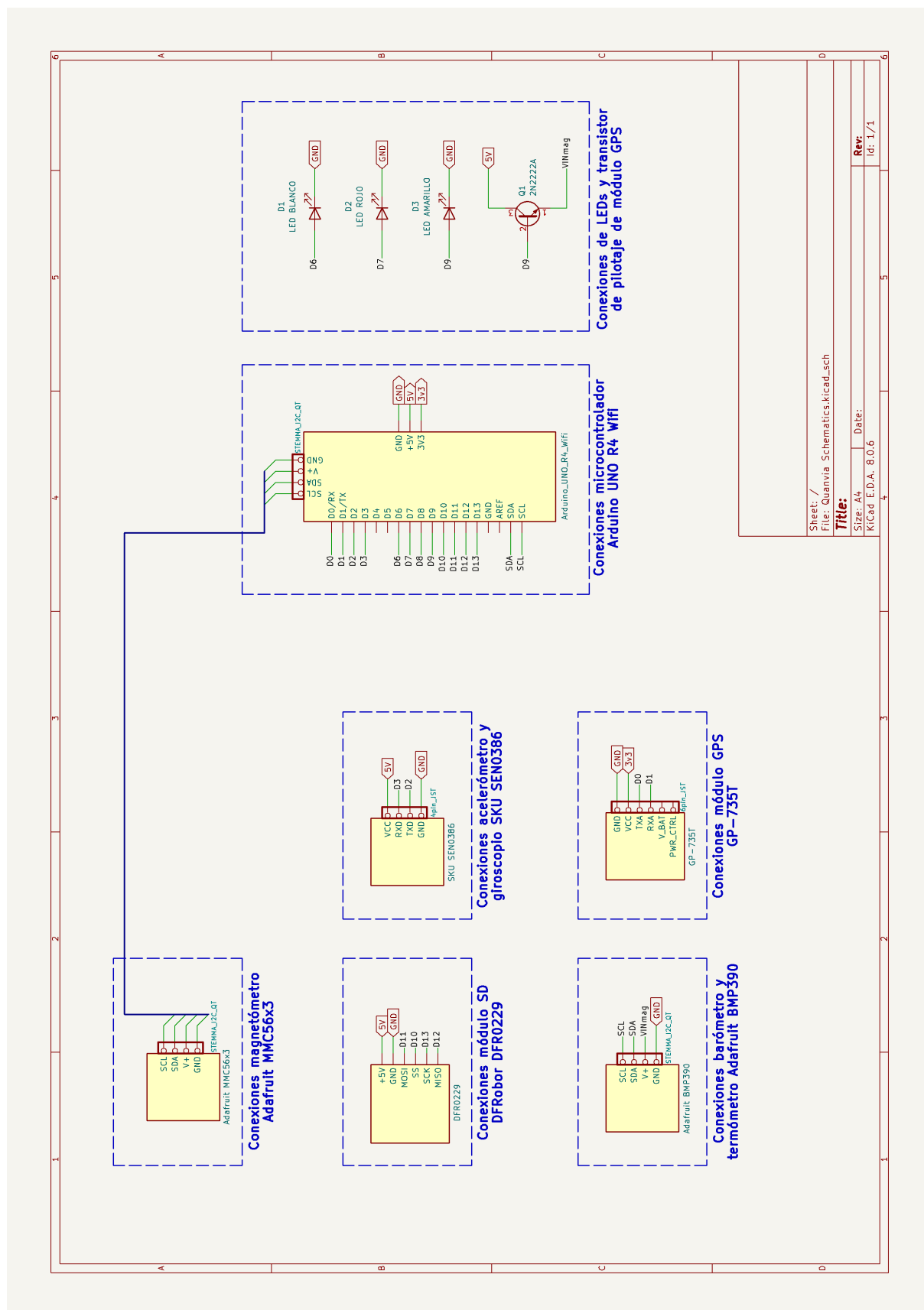


Figura 2: Rutas de test

A. Esquema de conexiones



B. Información sobre los sensores

B.1. Magnetómetro: Adafruit MMC56x3

El magnetómetro Adafruit MMC56x3 mide el valor del campo magnético en tres ejes (X , Y , Z) en unidades de μT y es utilizado frecuentemente en aplicaciones de navegación y orientación gracias a su alta precisión.

B.1.1. Especificaciones principales

- Rango de medición: $[-3000, 3000] \mu T$
- Resolución: $0,01 \mu T$
- Precisión del encabezado (*heading*): $\pm 1^\circ$
- Ruido total: $2 \mu T$ RMS
- Frecuencia de muestreo máxima: 1kHz
- Consumo en modo activo: 0,75 mA a 1,3 mA según configuración
- Corriente en modo apagado: $1 \mu A$
- Temperatura de operación: $[-40^\circ C, +85^\circ C]$

B.1.2. Cálculo de orientación (Heading)

El sensor permite calcular el ángulo de orientación respecto al norte magnético (en grados sexagesimales) usando las componentes X y Y del campo magnético. La fórmula utilizada para calcular el *heading*, en caso de que el sentido positivo del eje Z sea hacia arriba, es la siguiente:

$$heading = \frac{atan\left(\frac{B_y}{B_x}\right) \times 180}{\pi} \quad (1)$$

donde B_x y B_y son los valores del campo magnético en los ejes X y Y , respectivamente.

Si el sentido positivo del eje Z apunta hacia el suelo, la fórmula a utilizar varía ligeramente:

$$heading = -1 \times \frac{atan\left(\frac{B_x}{B_y}\right) \times 180}{\pi} \quad (2)$$

Es importante indicar que el *heading* se calcula respecto a la dirección positiva del eje Y . Si se necesita cambiar, basta con sumar o restar los grados necesarios. Con el montaje actual, el *heading* se calcula respecto a la dirección positiva del eje X , simplemente restando 90° al valor calculado con la fórmula 1.

Por último, se debe sumar (o restar en caso de que sea hacia el Oeste) la declinación magnética, que depende del lugar geográfico en el que nos localicemos. Este valor puede encontrarse fácilmente en esta [web](#) si conocemos las coordenadas del lugar en el que nos encontramos.

B.1.3. Calibración

Este tipo de sensores son muy sensibles a la presencia de elementos metálicos en su entorno, que pueden deformar los campos que trata de medir, así como a las interferencias de los campos artificiales creados por componentes electrónicos o por el propio sensor. Por este motivo, es necesario realizar una calibración del sensor que permita calcular como afectan los ya mencionados elementos a sus mediciones.

Es recomendable que la calibración se realice una vez el sensor haya sido montado en su localización definitiva, ya que una calibración que se realice para corregir las interferencias de un entorno específico, es altamente probable que pierda su utilidad si variamos sus condiciones, *e.g.* si montamos el sensor al lado de una batería y lo calibramos, pero luego esa batería se mueve a otro lugar, los parámetros de la calibración serán distintos.

Para el proceso de calibración pueden seguirse los pasos descritos en este [vídeo](#) o en esta [guía](#). El método más simple consiste en descargar el software MotionCal y emplear alguno de los códigos de Arduino que sea capaz de enviarle, en el formato correcto, los datos del magnetómetro (en la propia librería del sensor podemos encontrar ejemplos).

Si cargamos el programa a la placa Arduino y cerramos la terminal de la comunicación serial, desde el software de *MotionCal* podremos comunicarnos al puerto donde esté conectado el controlador. Al hacerlo, vemos como comienza a plotearse puntos de color rojo. El objetivo es girar el sensor en todas las direcciones posibles, intentando que los puntos dibujen una esfera lo más poblada posible. Una vez haya suficientes puntos, se saca una captura de las matrices (3×1) y (3×3) de la parte derecha, como las de la siguiente imagen:

Magnetic Offset		
-10.62		
-26.90		
-40.28		
Magnetic Mapping		
+0.986	-0.003	-0.004
-0.003	+0.998	-0.001
-0.004	-0.001	+1.016

Figura 3: Matrices de calibración del magnetómetro

Estos valores se corresponden a la corrección del offset de las medidas o de *hard-iron* en el caso de la matriz de tamaño (3×1) y la de corrección de escala o *soft-iron* en el caso de la (3×3) . Esta última matriz debería ser más o menos simétrica, con valores cercanos a 1 en su diagonal. Una vez conocidos los valores, basta aplicar la siguiente fórmula sobre las medidas del magnetómetro para obtener los valores calibrados:

$$m_{cal} = SI_{3 \times 3} \times [\tilde{m} - HI_{3 \times 1}] \quad (3)$$

donde m_{cal} es un vector de tamaño 3×1 con las 3 medidas del magnetómetro ya calibradas, SI es la matriz 3×3 y HI la 3×1 calculadas por el programa de calibración,

y $\tilde{m}$ es el vector de 3 elementos con las medidas en bruto del sensor. Si aplicamos esta transformación sobre las medidas en el código que permitía comunicarse con el software *MotionCal*, deberíamos ver como la esfera que se dibuja ahora es mucho más regular que antes y que los valores que se obtienen en las matrices de calibración son muy cercanos a cero, a excepción de en la diagonal de la matriz *soft – iron*, que deben ser cercanos a 1.

También es importante para el proceso de calibración mencionar que la librería del sensor cuenta con una función, que debe ser ejecutada en el arranque, que hace que pasen corrientes elevadas durante un breve período de tiempo por algunos de los componentes del sensor, con el objetivo de purgar posibles corrientes parásitas que hayan quedado en sus partes metálicas y que podrían provocar campos magnéticos que afectasen a sus medidas.

Utiliza el software *MotionCal* para realizar la calibración. Es necesario ajustar el *offset* de cada eje sumando los valores obtenidos por el sensor durante la calibración.

B.1.4. Funciones adicionales

- **Degaussing automático:** El magnetómetro incluye una función de *SET/RESET* que elimina el error causado por cambios en el campo magnético nulo debido a la temperatura y campos magnéticos externos. Esta función permite al sensor restablecerse automáticamente frente a magnetizaciones residuales.
- **Modo de lectura continua:** El sensor puede configurarse en un modo continuo que permite adquirir datos a distintas frecuencias, desde 1Hz hasta 1000Hz, lo que resulta útil en aplicaciones de alto rendimiento donde se requiere una lectura continua del campo magnético.
- **Medición de temperatura:** El dispositivo también ofrece la posibilidad de leer la temperatura **interna**.

B.1.5. Consideraciones de diseño

Es importante mantenerlo alejado de fuentes de campos magnéticos perturbadores (como bobinas o cables de alimentación) y evitar el uso de materiales que puedan magnetizarse alrededor del dispositivo. Además, se recomienda el uso de un condensador de desacople de $2,2 \mu F$ cercano al pin de alimentación.

B.1.6. Modo de operación

El sensor puede operar en dos modos:

- **Modo de medición puntual:** Cada medición se realiza bajo demanda enviando un comando I2C. Ideal para aplicaciones donde se necesita una lectura periódica, pero no continua.
- **Modo continuo:** El sensor realiza mediciones a intervalos definidos por la frecuencia de salida (ODR). Esto reduce la carga de procesamiento del microcontrolador, ya que el sensor almacena los resultados y se pueden leer posteriormente.

B.1.7. Enlaces de interés

- [Página del sensor en la web del fabricante](#)
- [Calibración con MotionCal](#)
- [Librería GitHub](#)

B.2. Sensor Barométrico y de Temperatura: BMP390

El BMP390 es un sensor de presión barométrica y temperatura de alta precisión, diseñado para aplicaciones de seguimiento de altitud y localización. Su bajo consumo energético y su alta precisión lo convierten en una opción ideal para dispositivos móviles y portátiles.

B.2.1. Especificaciones principales

- Rango de presión: [300, 1250] hPa
- Rango de temperatura: $[-40^{\circ}C, +85^{\circ}C]$
- Precisión absoluta: $\pm 0,50$ hPa (a 300 hPa a 1100 hPa y $T = 0^{\circ}C$ a $65^{\circ}C$)
- Precisión relativa: $\pm 0,03$ hPa (equivalente a ± 25 cm en altitud)
- Consumo de corriente: $3,2 \mu A$ (a 1 Hz)
- Frecuencia de muestreo máxima: 200 Hz
- Interfaz: I²C y SPI

B.2.2. Modo de operación

El BMP390 puede operar en tres modos:

- **Modo de sueño:** No se realizan mediciones.
- **Modo normal:** Ciclo automático entre períodos de medición y períodos de espera.
- **Modo forzado:** Se realiza una única medición y luego se regresa al modo de sueño.

B.2.3. Funciones adicionales

- **FIFO integrado:** Capacidad de 512 bytes que ayuda a reducir el consumo de energía y prevenir la pérdida de datos.
- **Ajuste de oversampling:** Opciones que van desde ultra-bajo consumo hasta la máxima resolución, permitiendo adaptarse a las necesidades de la aplicación.
- **Filtro IIR:** Minimiza las perturbaciones a corto plazo en los datos de salida, como los causados por cambios bruscos en el entorno.

B.2.4. Consideraciones de diseño

Se recomienda un diseño que minimice la exposición a fuentes de interferencia. Es aconsejable utilizar un condensador de desacople cerca de los pines de alimentación.

B.2.5. Enlaces de interés

- [Página del sensor en la web del fabricante](#)
- [Librería GitHub](#)

B.3. Acelerómetro y Giroscopio: SKU SEN0386

El módulo SKU SEN0386 integra giroscopios de alta precisión y un acelerómetro, junto con un microprocesador de alto rendimiento. Este dispositivo utiliza algoritmos avanzados de filtrado y solución dinámica de Kalman para determinar la actitud del módulo en tiempo real, lo que permite un rendimiento óptimo incluso en entornos dinámicos.

B.3.1. Especificaciones principales

- **Voltaje de operación:** 3,3 V – 5 V
- **Consumo de corriente:** < 40 mA
- **Tamaño:** 51,33 × 61,0 mm
- **Rango(s):**
 - Aceleración: $\pm 16\ g$
 - Velocidad angular: $\pm 2000\ ^\circ/s$
 - Ángulo de actitud: $\pm 180^\circ$

B.3.2. Funciones adicionales

- **Solucionador de gestos integrado:** Permite obtener actitudes precisas en entornos dinámicos combinando algoritmos de filtrado dinámico de Kalman.
- **Tecnología de filtrado digital avanzada:** Reduce el ruido de medición y mejora la precisión, logrando una precisión de medida estática de hasta $0,05^\circ$ (dinámica $0,1^\circ$).
- **Interfaz TTL:** Compatible con sistemas embebidos de 3.3V/5V para una fácil conexión.

B.3.3. Enlaces de interés

- [Página del sensor en la wiki de DFRobot](#)
- [Librería GitHub](#)

B.4. Módulo GPS: GP-735T

El GP-735T es un módulo de antena GPS de alto rendimiento, diseñado para facilitar la integración en diversas aplicaciones. Utiliza un chip de séptima generación de u-blox y es compatible con la biblioteca TinyGPS para leer y decodificar mensajes NMEA.

B.4.1. Especificaciones principales

- **Tipo de receptor:** 56 canales, frecuencia L1, código C/A
- **Precisión horizontal:**
 - $\leq 2,5\text{m}$ (autónomo)
 - $\leq 2,0\text{m}$ (WAAS)
- **Precisión de velocidad:**
 - $\leq 0,1\text{m/s}$ (velocidad)
 - $\leq 0,5^\circ$ (rumbos)
- **Consumo de corriente:** 37 mA (modo de seguimiento continuo)
- **Voltaje de operación:** 3,1 V – 5,5 V
- **Interfaz:** UART (TTL) y USB
- **Tamaño:** 8 mm $\times$ 35 mm $\times$ 6,55 mm
- **Temperatura de operación:** $[-40^\circ\text{C}, +85^\circ\text{C}]$

B.4.2. Funciones adicionales

- **Protocolos NMEA:** Compatible con NMEA 0183 versión 2.3, soportando mensajes como GPGLL, GPGLL, GPGSA, GPGSV, GPRMC y GPVTG.
- **Alimentación controlada por GPIO:** Permite encender o apagar el módulo a través de un pin de control de potencia.
- **Sensibilidad de seguimiento:** Hasta -162 dBm , lo que mejora el rendimiento en condiciones de señal débil.

B.4.3. Conexión y configuración

El módulo se conecta a través de UART utilizando los siguientes parámetros de comunicación:

- Velocidad de transmisión: 38400 bps (N-8-1)
- Mensajes NMEA por defecto: GGA, GLL, GSA, GSV, RMC, VTG

B.4.4. Consideraciones de diseño

- Mantener el módulo alejado de interferencias electromagnéticas.
- Asegurarse de que el módulo esté en un lugar con buena visibilidad al cielo para un mejor rendimiento del GPS.

B.4.5. Enlaces de interés

- [Página del sensor en la web del fabricante](#)
- [Librería TinyGPS en GitHub](#)

B.5. Módulo de Tarjeta SD: DFR0229

El módulo DFR0229 es un módulo de tarjeta MicroSD (TF) diseñado para su uso con Arduino. Este módulo permite expandir significativamente las capacidades de almacenamiento de datos de un Arduino, permitiendo registrar datos de sensores en formato CSV, así como almacenar audio, video y gráficos.

B.5.1. Especificaciones

- **Voltaje de funcionamiento:** 5 V
- **Tamaño:** 20 mm × 28 mm
- **Interfaz:** SPI
- **Compatibilidad:** MicroSD (TF)
- **Compatibilidad con Arduino:** Totalmente compatible con Arduino UNO/Mega y con la placa de expansión DFRobot IO Expansion Shield V5.

B.5.2. Funciones adicionales

- **Almacenamiento de datos:** Ideal para aplicaciones de registro de datos, permite almacenar información recogida por diversos sensores en formato CSV.
- **Compatibilidad SPI:** La interfaz SPI garantiza una comunicación rápida y eficiente entre el módulo y el microcontrolador.

B.5.3. Consideraciones de diseño

- Utilizar una tarjeta MicroSD (TF) adecuada y formateada correctamente en formato FAT.
- Proporcionar la alimentación adecuada de 5 V.

B.5.4. Enlaces de interés

- [Página del sensor en la wiki de DFRobot](#)
- [Librería GitHub](#)